

Bramble Emulator: Technical Reference Manual

Development Documentation

March 22, 2026

Contents

Introduction	5
1 Part 1: Architecture Overview and Design Philosophy	6
1.1 1.1 Project Summary	6
1.1.1 1.1.1 Project Statistics	6
1.2 1.2 Design Goals	6
1.3 1.3 System Characteristics	7
1.4 1.4 Execution Modes	8
1.4.1 1.4.1 Cooperative ARM Mode (default)	8
1.4.2 1.4.2 Threaded ARM Mode (-cores N or -cores auto)	9
1.4.3 1.4.3 RISC-V Mode (-arch rv32 or auto-detected)	9
1.4.4 1.4.4 Cortex-M33 Mode (-arch m33 or auto-detected)	10
1.5 1.5 Boot Sequences	10
1.5.1 1.5.1 RP2040 Boot	10
1.5.2 1.5.2 RP2350 RISC-V Boot (Picobin)	11
1.5.3 1.5.3 RP2350 ARM Boot	12
2 Part 2: Building and Running	13
2.1 2.1 Prerequisites	13
2.2 2.2 Building	13
2.2.1 2.2.1 Quick Build	13
2.2.2 2.2.2 Explicit CMake Build	14
2.2.3 2.2.3 Build Outputs	14
2.3 2.3 Running Tests	14
2.4 2.4 Running Firmware	15
2.4.1 2.4.1 Basic Usage	15
2.4.2 2.4.2 Architecture Selection	16
2.4.3 2.4.3 Multi-Core and Persistence	16
2.4.4 2.4.4 GDB Debugging	16
2.4.5 2.4.5 Networking and Multi-Device	16
2.4.6 2.4.6 Developer Tools	17
3 Part 3: Command-Line Reference	18
3.1 3.1 Usage Syntax	18
3.2 3.2 Architecture Selection	18
3.3 3.3 Core Options	18
3.4 3.4 Developer Tools	19
3.5 3.5 Storage Options	21
3.6 3.6 Networking and Multi-Device Options	22
4 Part 4: Memory Maps	24
4.1 4.1 RP2040 Memory Map	24
4.1.1 4.1.1 RP2040 Dual-Core RAM Layout	25
4.2 4.2 RP2350 Memory Map (Differences from RP2040)	25

4.3	4.3 Peripheral Base Addresses (Complete Table)	27
5	Part 5: RP2040 ARM CPU Engine	33
5.1	5.1 CPU State Structure (cpu_state_t)	33
5.2	5.2 Per-Core State (cpu_state_dual_t)	33
5.3	5.3 Instruction Dispatch	34
5.4	5.4 Instruction Set (Complete)	34
5.4.1	Thumb-1 (16-bit) Instructions	34
5.4.2	Thumb-2 (32-bit) Instructions (Available in M33 Mode)	35
5.5	5.5 Exception Handling	36
5.5.1	Exception Entry	36
5.5.2	Exception Return	36
5.5.3	Double-Fault Lockup	37
5.5.4	Interrupt Delivery Conditions	37
5.5.5	Priority Model	37
5.6	5.6 Timing Model	37
6	Part 6: RP2350 RISC-V CPU Engine	39
6.1	6.1 CPU State (rv_cpu_state_t)	39
6.2	6.2 Standard CSRs	39
6.3	6.3 Hazard3-Specific CSRs	41
6.4	6.4 Trap Handling	41
6.4.1	Trap Entry (rv_trap_enter)	41
6.4.2	Trap Return (MRET)	42
6.4.3	Trap Causes	42
6.5	6.5 CLINT Interrupt Controller	42
6.6	6.6 RISC-V Instruction Cache	43
6.7	6.7 ROM Function Table	43
6.8	6.8 Picobin IMAGE_DEF Parser	44
6.8.1	Block Structure	44
6.8.2	Item Encoding	44
6.8.3	Item Types	44
6.8.4	Example: RISC-V IMAGE_DEF	45
7	Part 7: RP2350 Peripherals	46
7.1	7.1 TICKS (0x40108000)	46
7.2	7.2 POWMAN (0x40100000)	46
7.3	7.3 QMI (0x400D0000)	47
7.4	7.4 OTP (0x40120000 / 0x40130000)	47
7.5	7.5 BOOTRAM (0x400E0000)	47
7.6	7.6 TIMER1 (0x400B8000)	47
7.7	7.7 Other RP2350 Peripherals	48
8	Part 8: SIO and Inter-Core Communication	49
8.1	8.1 SIO Base (0xD0000000)	49
8.2	8.2 Inter-Core FIFO	49
8.3	8.3 RP2350 Hart Launch Mailbox	50

9	Part 9: Debugging	51
9.1	9.1 GDB Remote Debugging	51
9.1.1	Setup	51
9.1.2	Supported RSP Commands	51
9.1.3	Architecture-Aware Registers	52
9.1.4	Conditional Breakpoints	52
9.1.5	Watchpoint Stop Replies	52
9.1.6	Limits	52
9.2	9.2 Debug Output Flags	53
10	Part 10: Storage and Persistence	54
10.1	10.1 Flash Persistence	54
10.2	10.2 SD Card	54
10.3	10.3 eMMC	55
10.4	10.4 FUSE Mount	55
11	Part 11: Networking, WiFi, and Multi-Device	56
11.1	11.1 UART-to-TCP Bridge	56
11.2	11.2 Wire Protocol (Multi-Instance IPC)	56
11.3	11.3 CYW43439 WiFi Emulation	56
11.3.1	TAP Bridge	57
12	Part 12: Performance	58
12.1	12.1 ARM Instruction Cache	58
12.2	12.2 ARM JIT Basic Block Compilation (-jit)	58
12.3	12.3 RISC-V Instruction Cache	58
12.4	12.4 Host Threading	59
13	Part 13: Tested Firmware	60
13.1	13.1 RP2040 Firmware	60
13.2	13.2 RP2350 Firmware	60
14	Part 14: Design Decisions and Trade-Offs	62
14.1	14.1 Correctness Over Performance	62
14.2	14.2 Synchronous DMA	62
14.3	14.3 Immediate PIO Execution	62
14.4	14.4 Tri-Architecture Shared Peripheral Bus	62
14.5	14.5 Output Separation	63
14.6	14.6 Flash Initialized to 0xFF	63
15	Part 15: Repository Structure	64
16	Part 16: Version History	66
17	Part 17: External References	68

Introduction

Bramble is a high-fidelity development and debugging tool for the Raspberry Pi RP2040 and RP2350 microcontroller ecosystem. Unlike high-level simulators, Bramble focuses on register-level accuracy, allowing unmodified firmware binaries—including those from the Pico SDK, MicroPython, CircuitPython, and littleOS—to execute in a controlled, virtualized environment.

The emulator supports **tri-architecture** execution:

- **RP2040 Cortex-M0+** (ARMv6-M Thumb-1, 65+ instructions)
- **RP2350 Cortex-M33** (ARMv8-M Mainline, full Thumb-2 ISA)
- **RP2350 Hazard3 RISC-V** (RV32IMAC, 93 instructions)

Architecture is auto-detected from UF2 family ID, ELF machine type, or RP2350 picobin IMAGE_DEF blocks. All three architectures share a unified peripheral infrastructure covering 30+ emulated hardware modules.

This manual serves as the primary technical reference for the emulator’s architecture, instruction set coverage, peripheral implementation, boot sequences, debugging facilities, and developer tooling.

Version 0.43.0 — 300 tests, 22,663 source lines, zero compiler warnings.

Chapter 1

Part 1: Architecture Overview and Design Philosophy

1.1 1.1 Project Summary

Bramble is a from-scratch emulator for Raspberry Pi RP2040 and RP2350 microcontrollers written in C99 with POSIX extensions. It executes real firmware—UF2 and ELF binaries produced by the Pico SDK, MicroPython, CircuitPython, littleOS, and other projects—without modification.

The emulator provides **tri-architecture** support:

- **RP2040 Cortex-M0+** (-arch m0+): 65+ Thumb-1 instructions with O(1) dispatch, JIT basic block compilation, and full dual-core emulation.
- **RP2350 Cortex-M33** (-arch m33): Full Thumb-2 ISA reusing the existing ARM engine with BASEPRI, M33 CPUID, and 520 KB SRAM.
- **RP2350 Hazard3 RISC-V** (-arch rv32): Complete RV32IMAC (93 instructions) with CLINT interrupt controller, SDK-compatible bootrom, and picobin boot.

All three architectures share the same peripheral infrastructure, storage subsystem, networking stack, and developer tools.

1.1.1 Project Statistics

Metric	Value
Source lines	22,663 (42 .c files)
Header files	40 .h files
Test suite	300 tests across 57 categories
Compiler warnings	Zero (-Wall -Wextra -pedantic)
Tested firmware	MicroPython, CircuitPython, littleOS (RP2040 + RP2350)
Version	0.43.0

1.2 1.2 Design Goals

1. **Boot real firmware unmodified.** Any UF2 or ELF produced by the Pico SDK, MicroPython, CircuitPython, or compatible toolchains should load and execute without patching.

2. **Register-level peripheral fidelity.** Every emulated peripheral reproduces the documented register layout and behavior from the RP2040/RP2350 datasheets. Firmware that reads status bits, sets interrupt masks, or polls FIFO levels gets correct values.
3. **Automatic architecture detection.** UF2 family IDs (0xE48BFF56 for RP2040, 0xE48BFF59 for RP2350 ARM, 0xE48BFF5A for RP2350 RISC-V), ELF machine types (EM_ARM = 40, EM_RISCV = 243), and picobin IMAGE_DEF blocks automatically select the correct execution engine and memory map.
4. **Interactive debugging.** The integrated GDB stub supports breakpoints, watchpoints, conditional breakpoints, dual-core thread selection, and architecture-aware register layouts (17 registers for ARM, 33 for RISC-V).
5. **Multi-device and network support.** UART-to-TCP bridging, Unix socket IPC between Bramble instances, SPI-attached SD card and eMMC, and CYW43 WiFi emulation with TAP bridging enable complex system-level testing.
6. **Developer tooling.** 18 built-in tools: semihosting, code coverage, hotspot analysis, instruction trace, call graph, VCD waveform export, IRQ latency measurement, stack checking, bus logging, scripted I/O, expected output matching, memory watch, fault injection, cycle profiling, and memory heatmap.

1.3 1.3 System Characteristics

Feature	Details
Language	C99 with POSIX extensions (pthreads, sockets, mmap)
Build system	CMake 3.10+; links -lm -lpthread; optional -lfuse3
Host platforms	Linux (primary), macOS (compatible)
RP2040 ARM ISA	65+ Thumb-1 + 32-bit BL/MSR/MRS/DSB/DMB/ISB. See Section 5.4 for full listing.
RP2350 Thumb-2 ISA	Full ARMv8-M: MOVW/MOVT, SDIV/UDIV, CLZ, MLA/MLS, SMULL/UMULL/SMLAL/UMLAL, BFI/BFC, UBFX/SBFX, B.W, TBB/TBH, wide LD/ST. See Section 5.4.2.
RISC-V ISA	RV32I (37) + M (8) + A (11) + C (36) + Zicsr (6) = 93 instructions. See Section 6.
RP2040 memory	4 MB flash array (2 MB default), 264 KB SRAM (6 banks), 16 KB ROM
RP2350 memory	4 MB flash array, 520 KB SRAM (10 banks + scratch), 32 KB ROM

Feature	Details
Peripherals	30+ modules. See Section 4.3 for complete peripheral address table.
Firmware formats	UF2 (family ID auto-detect), ELF32 (ARM + RISC-V), picobin IMAGE_DEF
Debugging	GDB RSP; 16 breakpoints, 16 watchpoints; conditional BPs; dual-core threads; ARM + RISC-V registers
Clock model	Configurable -clock <MHz> (default 1; RP2040: 125; RP2350: 150). ARMv6-M cycle timing per DDI 0484C.
ARM performance	64K icache + optional 16384-entry JIT (-jit, ~1.5x speedup)
RISC-V performance	64K icache for flash/ROM; 99.97%+ hit rates observed
Storage	Flash persistence, FUSE mount, SPI SD card (SDHC), SPI eMMC, FAT12/FAT16
Networking	UART-to-TCP, Unix socket wire protocol, CYW43 WiFi gSPI + TAP bridge
Output model	Firmware output (UART TX, USB CDC) on stdout; all diagnostics on stderr

1.4 Execution Modes

Bramble supports four execution modes, selected by architecture and CLI flags:

1.4.1 Cooperative ARM Mode (default)

This is the primary execution mode for RP2040 and RP2350 Cortex-M33 firmware:

1. `dual_core_step()` executes one instruction on each active core via round-robin scheduling. It calls `cpu_bind_core_context()` to swap the active register file into the global `cpu_struct`, executes `cpu_step()`, then calls `cpu_unbind_core_context()` to save back.
2. `pio_step()` advances all enabled PIO state machines (across all 3 PIO blocks). Each SM's fractional clock divider (16.8 fixed-point accumulator) is checked; SMs only step when their accumulator overflows.
3. `usb_step()` advances the USB host enumeration state machine. It transitions through states: IDLE, BUS_RESET, GET_DEVICE_DESCRIPTOR, SET_ADDRESS, GET_CONFIG_DESCRIPTOR, SET_CONFIGURATION, SET_CONTROL_LINE_STATE, ACTIVE. Each transition is paced by the firmware's response (writing AVAILABLE to endpoint `buf_ctrl`).

4. `timing_tick()` accumulates CPU cycles. When `cycle_accumulator >= cycles_per_us`, it converts to microseconds and calls `timer_tick()`, `rtc_tick()`, and `systick_tick_for_core()` for each active core.
5. Stdin polling (`uart_stdin_poll()`), network bridge polling (`net_bridge_poll()`), and wire protocol polling (`wire_poll()`) occur every 1024 steps.

1.4.2 1.4.2 Threaded ARM Mode (-cores N or -cores auto)

When threading is enabled, each emulated core gets its own host pthread:

1. Each thread acquires a global mutex (“big lock”) before executing guest instructions.
2. The thread executes `step_quantum` instructions (default 64, configurable via `-thread-quantum`), then releases the lock.
3. When a core executes WFI or WFE, it releases the lock and sleeps on a condition variable. `corepool_wake_cores()` signals the condvar when an interrupt becomes pending (NVIC `set_pending`, SysTick tick, ICSR `PendSV/SysTick set`, Core 1 launch).
4. The main thread handles I/O polling (stdin, network, wire, WiFi TAP), watchdog checking, and periodic storage flush.
5. A file-based core pool registry at `/tmp/bramble-corepool.reg` tracks running instances. `-cores auto` queries this registry to determine optimal allocation (total host CPUs minus already-claimed cores).

1.4.3 1.4.3 RISC-V Mode (-arch rv32 or auto-detected)

The RISC-V execution path runs independently from the ARM CPU engine:

1. Two `rv_cpu_state_t` structs represent the dual Hazard3 harts. Each has 32 general-purpose registers (x0 hardwired to zero), a PC, 4096 CSR slots, 64-bit cycle/instruction counters, LR/SC reservation state, and a bus pointer.
2. An `rv_membus_state_t` provides the RP2350 memory landscape: 520 KB SRAM, 32 KB ROM, CLINT registers, and RP2350-specific peripherals. Unhandled addresses fall through to the shared RP2040 peripheral bus (`mem_read32/mem_write32`), which provides UART, SPI, I2C, GPIO, Timer, PWM, DMA, PIO, USB, etc.
3. Each iteration of the main loop:
 - Checks for ROM function interception (`rv_rom_intercept()`). If the PC is at a ROM function stub (0x0400–0x0437), the operation is performed natively in C and the PC advances to the return address.
 - Steps hart 0 (if not WFI): `rv_cpu_step()` fetches via the instruction cache (`rv_icache_lookup`), decodes (compressed if bits [1:0] != 11), and executes.
 - Steps hart 1 (if not halted and not WFI).
 - Advances CLINT timer: `rv_clint_tick(&bus.clint, 1)` accumulates cycles and increments `mtime` when a microsecond boundary is crossed. Also ticks TIMER1 via `rp2350_timer1_tick()`.

- Checks and delivers interrupts: `rv_clint_check_interrupts()` computes mip from hardware state (timer compare, MSIP, external pending), checks `mstatus.MIE` and `mie`, and calls `rv_trap_enter()` for the highest-priority deliverable interrupt. Priority order: MEIP (external, cause `0x8000000B`) > MSIP (software, cause `0x80000003`) > MTIP (timer, cause `0x80000007`).
- Checks hart 1 launch mailbox: if `rv_membus_check_hart1_launch()` detects a pending launch (firmware wrote to SIO at `0xD00001CC`), resets hart 1 with the specified entry PC, SP (x2), and argument (a0/x10).
- Checks for RISC-V semihosting: if hart 0's mcause is `MCAUSE_BREAKPOINT` and `x[10]` (a0) is `0x20026`, triggers `SYS_EXIT`.
- Polls stdin, network, and wire every 1024 steps.

1.4.4 Cortex-M33 Mode (-arch m33 or auto-detected)

The M33 mode runs on the existing ARM CPU engine with an overlay:

1. `m33_init_overlay()` sets the CPUID to `0x410FD210` (Cortex-M33 r0p0) and initializes BASEPRI to 0.
2. A static 520 KB SRAM buffer is allocated and installed via `mem_set_ram_ptr()`. The `cpu_bind_core_context()` function checks `membus_rp2350_mode` and uses the 520 KB pointer instead of the RP2040's 264 KB.
3. `membus_rp2350_mode = 1` enables RP2350-specific peripheral routing in the shared membus: `SYSINFO` returns RP2350 CHIP_ID, RP2350 peripherals are routed, and ROM writes are silently suppressed.
4. The MSR/MRS handlers in `instructions.c` support BASEPRI (SYSm 0x11) and BASEPRI_MAX (SYSm 0x12, write-if-greater semantics).
5. All Thumb-2 instructions are already implemented in `thumb32.c` (called when bits [15:11] of the first halfword indicate a 32-bit encoding). This includes MOVW/MOVT, SDIV/UDIV, CLZ, MLA/MLS, SMULL/UMULL/UMLAL/SMLAL, BFI/BFC, UBFX/SBFX, B.W, TBB/TBH, wide loads/stores, and data processing with shifted registers.

1.5 Boot Sequences

1.5.1 RP2040 Boot

```
cpu_init()
└─ cpu.flash[] initialized to 0xFF (erased state)
load_uf2() or load_elf()
└─ Firmware written to cpu.flash[] starting at offset 0
   (target address 0x10000000)
└─ UF2 family ID detected; loader_detected_arch() set
Flash persistence restore
└─ If -flash <path>: non-firmware 4KB sectors restored from file
Boot2 detection
```

- └ First flash word checked for valid ARM vector (SP value)
- └ If valid: Core 0 PC = 0x10000000 (boot2 entry)
- └ If invalid or -no-boot2: Core 0 PC = 0x10000100

Vector table read

- └ SP from flash[boot_offset + 0]
- └ Reset vector from flash[boot_offset + 4]

Core 0 begins execution

Core 1 starts halted

- └ Launched by firmware via SIO FIFO protocol
(write 0, 0, 1, VTOR, SP, entry to FIFO)

1.5.2 1.5.2 RP2350 RISC-V Boot (Picobin)

load_uf2()

- └ UF2 family ID 0xE48BFF57 or 0xE48BFF5A detected
- └ Blocks loaded into cpu.flash[] (up to 4MB offset)

rv_membus_init()

- └ 520KB SRAM zeroed
- └ CLINT initialized (mtimecmp = UINT64_MAX)
- └ RP2350 peripherals initialized

rv_bootrom_init()

- └ 32KB ROM populated with:
 - Reset vector at 0x0000 (JAL to boot code)
 - Trap handler at 0x0004 (C.J self, infinite loop)
 - Magic header at 0x0010: 'R', 'P', 0x02
 - Function table at 0x0100 (9 entries)
 - Lookup function at 0x0300 (RISC-V code)
 - ROM function stubs at 0x0400+ (RET instructions)
 - Boot code at 0x0020: LUI SP, ADDI SP, LUI GP,
ADDI t0(mtvec), CSRRW mtvec, LUI t0(flash), JALR t0

picobin_scan(cpu.flash, 4096)

- └ Scans first 4KB for marker 0xFFFFDED3
- └ Parses IMAGE_TYPE item (flags word):
 - bits[3:0] = image type (1=EXE)
 - bits[10:8] = CPU (0=ARM, 1=RISC-V)
 - bits[14:12] = chip (0=RP2040, 1=RP2350)
- └ Parses ENTRY_POINT item: PC and SP
- └ Parses VECTOR_TABLE item (ARM only): VTOR

If picobin found with valid entry:

- └ Hart 0 reset to picobin entry PC
- └ SP (x2) set to picobin entry SP

Else:

- └ Hart 0 reset to 0x00000000 (ROM bootrom)

Hart 1 starts halted

- └ Launched via SIO mailbox at 0xD00001C0

1.5.3 1.5.3 RP2350 ARM Boot

load_uf2()

- └ UF2 family ID 0xE48BFF59 or auto-detected

m33_init_overlay()

- └ nvic_cpuid_value = 0x410FD210

- └ 520KB SRAM allocated and installed

- └ membus_rp2350_mode = 1

- └ RP2350 peripherals initialized

Standard ARM vector table boot

- └ SP from flash[0], PC from flash[4] (Thumb bit stripped)

- └ Same as RP2040 boot but with M33 features active

Chapter 2

Part 2: Building and Running

2.1 2.1 Prerequisites

Required:

- C99 compiler (GCC 10+ or Clang 12+)
- CMake 3.10+
- POSIX-compatible OS (Linux or macOS)
- pthreads (included on all POSIX systems)
- libmath (-lm)

Optional:

- libfuse3-dev — for -mount filesystem passthrough support
- arm-none-eabi-gcc — to build RP2040 test firmware
- riscv32-unknown-elf-gcc — to build RP2350 RISC-V test firmware
- graphviz — to visualize -callgraph output
- gtkwave or pulseview — to view -gpio-trace VCD files

2.2 2.2 Building

2.2.1 Quick Build

```
./build.sh
```

The `build.sh` script auto-detects FUSE availability, creates a `build/` directory, runs CMake with Release configuration, and copies the `bramble` binary to the project root.

Build script flags:

Flag	Description
<code>--clean</code>	Remove existing build directory before building
<code>--no-fuse</code>	Disable FUSE even if libfuse3 is available
<code>--release</code>	Release build with optimizations (default)
<code>--debug</code>	Debug build with symbols, no optimization
<code>--help</code>	Show usage information

2.2.2 Explicit CMake Build

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
```

CMake options:

Option	Default	Description
CMAKE_BUILD_TYPE	Release	Build type (Release, Debug, RelWithDebInfo)
CMAKE_C_COMPILER	system cc	C compiler path (e.g., clang)
ENABLE_FUSE	auto-detect	Enable FUSE filesystem mount (ON, OFF)

2.2.3 Build Outputs

Binary	Description
bramble	Main emulator executable (copied to project root)
bramble_tests	Automated test suite
bramble_bench	Performance benchmarking tool

2.3 2.3 Running Tests

```
# Run via CTest (preferred)
ctest --test-dir build --output-on-failure

# Run directly
./build/bramble_tests
```

The test suite contains **300 tests** organized into 57 categories:

Category Group	Tests	Description
ARM instruction tests	~180	PRIMASK, SVC, RAM exec, dispatch, ADCS/SBCS/RSBS, shifts, branches, STMIA/LDMIA, MUL, BL, MSR/MRS, CMN, ADR, ADD/SUB SP

Category Group	Tests	Description
Peripheral tests	~60	Timer, GPIO, NVIC, SysTick, UART (Tx + Rx FIFO), SPI, I2C, PWM, DMA, PIO (execution + clock division), ADC (FIFO + round-robin), USB, RTC
Memory/loader tests	~20	Flash, RAM, SRAM alias, XIP cache, XIP aliases, UF2 loader, ELF loader, memory bus subword access
Exception tests	~10	Entry/return, pending IRQ delivery, nesting, HardFault, double-fault lockup
Dual-core tests	~10	RAM isolation, shared flash, context bind/unbind, spinlocks, FIFO, core pool, WFI, wire protocol
RISC-V tests	20	CPU init/reset, ADDI, LUI, ADD/SUB, BEQ, SW/LW, MUL/DIV, JAL/JALR, C.LI/C.ADDI, CSR mhartid, trap enter/return, CLINT timer/interrupt, membus SRAM, bootrom init, icache, peripherals (BOOTRAM, TIMER1, Hazard3 CSRs)
M33 tests	4	CPUID switching, BASEPRI read/write/BASEPRI_MAX, SDIV, MOVW

2.4 2.4 Running Firmware

2.4.1 Basic Usage

```
# RP2040 firmware (architecture auto-detected)
./bramble hello_world.uf2

# With real-time clock and interactive stdin
./bramble firmware.uf2 -clock 125 -stdin

# ELF firmware (auto-detected by .elf extension)
./bramble firmware.elf
```

2.4.2 Architecture Selection

```
# Explicit architecture
./bramble firmware.uf2 -arch m0+      # RP2040 Cortex-M0+
./bramble firmware.uf2 -arch m33      # RP2350 Cortex-M33
./bramble firmware.uf2 -arch rv32     # RP2350 Hazard3 RISC-V

# Auto-detected from UF2 family ID
./bramble pico2_firmware.uf2          # Detects RP2350
./bramble micropython_pico2_rv.uf2    # Detects RP2350 RISC-V
```

2.4.3 Multi-Core and Persistence

```
# Dual-core threaded with flash persistence
./bramble firmware.uf2 -cores 2 -clock 125 -flash storage.bin

# Auto-detect core count from pool
./bramble firmware.uf2 -cores auto

# FUSE mount flash filesystem
./bramble firmware.uf2 -flash storage.bin -mount /tmp/pico-fs
```

2.4.4 GDB Debugging

```
# Terminal 1: start emulator with GDB server
./bramble firmware.uf2 -gdb 3333

# Terminal 2: connect GDB
arm-none-eabi-gdb firmware.elf -ex "target remote :3333"

# For RISC-V firmware
riscv32-unknown-elf-gdb firmware.elf -ex "target remote :3333"
```

2.4.5 Networking and Multi-Device

```
# Bridge UART0 to TCP (connect with nc or minicom)
./bramble firmware.uf2 -net-uart0 9999 -stdin

# Wire two instances together
./bramble fw_sensor.uf2 -wire-uart0 /tmp/uart.sock -stdin # Terminal 1
./bramble fw_ctrl.uf2 -wire-uart0 /tmp/uart.sock -stdin # Terminal 2

# WiFi with TAP bridge
```



```
./bramble firmware.uf2 -wifi -tap tap0
```

SD card

```
./bramble firmware.uf2 -sdcard sdcard.img -sdcard-size 32
```

2.4.6 Developer Tools

Code coverage and hotspot analysis

```
./bramble firmware.uf2 -coverage cov.bin -hotspots 20
```

Instruction trace with symbols

```
./bramble firmware.uf2 -trace trace.bin -symbols firmware.elf
```

GPIO VCD trace for waveform viewer

```
./bramble firmware.uf2 -gpio-trace pins.vcd
```

Expected output matching (CI integration)

```
./bramble firmware.uf2 -expect golden.txt -timeout 5
```

Fault injection

```
./bramble firmware.uf2 -inject-fault flash_bitflip:1000000:0x10000100
```

Chapter 3

Part 3: Command-Line Reference

3.1 3.1 Usage Syntax

```
bramble <firmware.uf2|firmware.elf> [options]
```

The firmware path **must** be the first argument. All options follow.

3.2 3.2 Architecture Selection

Flag	Arguments	Description
-arch	m0+ or arm	RP2040 Cortex-M0+ (default if not auto-detected)
	m33	RP2350 Cortex-M33 (ARMv8-M Mainline, full Thumb-2)
	rv32 or riscv	RP2350 Hazard3 RISC-V (RV32IMAC)

Auto-detection priority:

1. **UF2 family ID** (if flags bit 13 set): 0xE48BFF56 -> M0+, 0xE48BFF59 -> M33, 0xE48BFF5A -> RV32.
2. **ELF machine type**: EM_ARM (40) -> M0+, EM_RISCV (243) -> RV32.
3. If neither present, defaults to M0+.

3.3 3.3 Core Options

Flag	Arguments	Description
-debug		Enable verbose CPU debug output for Core 0 / Hart 0
-debug1		Enable debug output for Core 1 / Hart 1
-asm		Show assembly instruction tracing
-status		Print periodic status updates (PC, SP, FIFO, core state)

Flag	Arguments	Description
-stdin		Route host stdin to active guest console (USB CDC if enumerated, else UART0). LF translated to CR for serial compatibility.
-gdb	[port]	Start GDB RSP TCP server (default port: 3333). Disables instruction limit and JIT.
-clock	<MHz>	CPU clock frequency in MHz. Default: 1 (fast-forward). Real RP2040: 125. Real RP2350: 150. Sets <code>timing_config.cycles_per_us</code> .
-cores	<N\ auto>	Number of active cores: 1, 2, or auto (queries core pool). Values > 1 enable host-threaded execution.
-thread-quantum	<N>	Guest instructions per lock hold in threaded mode. Range: 1–4096. Default: 64.
-no-boot2		Skip boot2 execution even if detected in flash. Core 0 starts at 0x10000100 instead of 0x10000000.
-debug-mem		Log every unmapped peripheral read/write to stderr. Useful for finding unimplemented peripherals.
-jit		Enable JIT basic block compilation for ARM flash/ROM code. 16384-entry cache, max 64 instructions per block. ~1.5x speedup.

3.4 3.4 Developer Tools

Flag	Arguments	Description
-semihosting		ARM: intercept BKPT #0xAB for SYS_WRITE0, SYS_EXIT, etc. RISC-V: intercept EBREAK with a0=0x20026 for SYS_EXIT.
-coverage	<file>	Write binary code coverage bitmap on exit. Each bit represents a 2-byte-aligned PC in flash+RAM range.

Flag	Arguments	Description
-hotspots	[N]	Print top N most-executed PCs on exit (default: 20). Combined with -symbols shows function names.
-trace	<file>	Write binary instruction trace: each entry is (PC, opcode, cycles).
-exit-code	<addr>	On halt, read uint32_t from this RAM address and use as process exit code.
-timeout	<seconds>	Kill emulator after N seconds. Exit code 124 (like timeout(1)).
-symbols	<elf>	Parse ELF .symtab/.strtab for function name resolution in hotspots, profiles, callgraphs, crash reports, and watch logs.
-callgraph	<file>	Write DOT-format call graph tracking BL/BLX calls. Open with dot -Tpng callgraph.dot -o cg.png.
-stack-check		Track per-core SP high-water mark. Reports peak stack depth on exit. Warns if SP approaches RAM base.
-irq-latency		Measure cycles from nvic_signal_irq() to exception handler entry. Reports min/avg/max per IRQ number on exit.
-log-uart		Log every UART TX/RX byte to stderr with hex and printable representation.
-log-spi		Log every SPI MOSI/MISO byte to stderr.
-log-i2c		Log every I2C DATA_CMD transaction to stderr.
-gpio-trace	<file>	Record all GPIO pin changes with cycle-accurate timestamps in Value Change Dump (VCD) format. Open with GTKWave or PulseView.
-script	<file>	Feed timestamped input from text file. Syntax: 100ms: uart0 "hello\n", 200ms: gpio25 1.
-expect	<file>	Capture stdout and compare against golden file on exit. Exit 0 if match, exit 1 with byte-level diff location if mismatch.

Flag	Arguments	Description
-watch	<addr[:len]>	Log every read/write to address range to stderr with PC and symbol context. Up to 8 simultaneous watch regions.
-inject-fault	<spec>	Schedule hardware faults at cycle counts. Specs: flash_bitflip:cycle:addr, ram_corrupt:cycle:addr, brownout:cycle.
-profile	<file>	Per-PC cycle accounting in CSV format. Combined with -symbols gives per-function timing breakdown.
-mem-heatmap	<file>	Track read/write frequency per 256-byte RAM block. CSV output for visualization.

3.5 3.5 Storage Options

Flag	Arguments	Description
-flash	<path>	Persistent flash storage file. On startup: firmware sectors preserved, non-firmware 4KB sectors restored from file. Write-through: every flash_range_erase/flash_range_program immediately syncs via flash_persist_sync(). On exit: full image saved.
-mount	<dir>	Mount flash FAT12/FAT16 filesystem as host directory via libfuse3. Requires build with ENABLE_FUSE=ON. Works with or without -flash: without, mount is volatile. Thread-safe via fuse_flash_mutex.
-mount-offset	<hex>	Flash byte offset of FAT boot sector. Default: auto-scanned (searches for 0x55AA signature + valid BPB geometry).

Flag	Arguments	Description
-sdcard	<path>	Attach file-backed SD card image on SPI bus. SDHC protocol, CSD v2.0, CID (product name "BRMSD"). Commands: CMD0/8/9/10/12/13/16/17/18/24/25/55/58, ACMD41.
-sdcard-spi	<0\ 1>	SPI bus for SD card (default: 1).
-sdcard-size	<MB>	SD card size in MB (default: 64).
-emmc	<path>	Attach file-backed eMMC image on SPI bus. CMD1 init, EXT_CSD via CMD8, sector addressing, product name "BRMMC".
-emmc-spi	<0\ 1>	SPI bus for eMMC (default: 0).
-emmc-size	<MB>	eMMC size in MB (default: 128).

3.6 Networking and Multi-Device Options

Flag	Arguments	Description
-net-uart0	<port>	Bridge UART0 TX/RX to TCP server socket. Connect with nc localhost <port> or minicom. Non-blocking I/O with TCP_NODELAY.
-net-uart1	<port>	Bridge UART1 TX/RX to TCP server socket.
-net-uart0-connect	<host:port>	Connect UART0 to remote TCP host (client mode).
-net-uart1-connect	<host:port>	Connect UART1 to remote TCP host (client mode).
-wire-uart0	<path>	Wire UART0 to peer Bramble instance via Unix domain socket. First instance creates socket (listen), second connects (auto-negotiation). UART TX on one arrives as UART RX on other.
-wire-uart1	<path>	Wire UART1 via Unix domain socket.
-wire-gpio	<path>	Wire GPIO pin state via Unix domain socket. Pin changes propagated between instances.

Flag	Arguments	Description
-wifi		Enable CYW43439 WiFi chip emulation (Pico W). gSPI protocol via PIO0 SM0 FIFO intercept.
-tap	<ifname>	Bridge CYW43 WLAN frames to host TAP interface. Implies -wifi. Auto-configures 192.168.4.1/24, IP forwarding, NAT masquerade. Requires sudo.

Chapter 4

Part 4: Memory Maps

4.1 4.1 RP2040 Memory Map

Start Address	End Address	Size	Region
0x00000000	0x00003FFF	16 KB	ROM (bootrom with function table)
0x10000000	0x103FFFFFF	4 MB	XIP Flash (2 MB default, 4 MB array)
0x11000000	0x113FFFFFF	4 MB	XIP NOALLOC alias (uncached read, allocate bypass)
0x12000000	0x123FFFFFF	4 MB	XIP NOCACHE alias (cache bypass, uncached access)
0x13000000	0x133FFFFFF	4 MB	XIP NO-CACHE_NOALLOC alias
0x14000000	0x1400001F	32 B	XIP Cache Control (CTRL, FLUSH, STAT, CTR_HIT, CTR_ACC, STREAM)
0x15000000	0x15003FFF	16 KB	XIP SRAM (cache memory usable as general SRAM)
0x18000000	0x18000FFF	4 KB	XIP SSI (flash SPI controller) + atomic aliases at +0x1000/+0x2000/+0x3000

Start Address	End Address	Size	Region
0x20000000	0x20041FFF	264 KB	SRAM (6 banks: 4x64KB + 2x4KB)
0x21000000	0x21041FFF	264 KB	SRAM alias (simple mirror, NOT atomic)
0x40000000	0x4006FFFF		APB Peripherals (see Section 4.3)
0x50000000	0x503FFFFF		AHB Peripherals (DMA, USB, PIO0, PIO1)
0xD0000000	0xD00001FF		SIO (single-cycle I/O: GPIO, FIFO, divider, interpolators, spinlocks)
0xE0000000	0xE000FFFF		PPB (NVIC, SysTick, SCB at 0xE000E000)

4.1.1 RP2040 Dual-Core RAM Layout

Start Address	End Address	Size	Owner
0x20000000	0x2001FFFF	128 KB	Core 0 private SRAM
0x20020000	0x2003FFFF	128 KB	Core 1 private SRAM
0x20040000	0x20041FFF	8 KB	Shared SRAM

In single-core mode, the full 264 KB is available as a flat address range via `cpu.ram[]`.

4.2 4.2 RP2350 Memory Map (Differences from RP2040)

Start Address	End Address	Size	Region	Note
0x00000000	0x00007FFF	32 KB	ROM	Larger than RP2040's 16 KB
0x20000000	0x20081FFF	520 KB	SRAM	10 banks x 64KB + 8KB scratch
0x400B0000	0x400B00FF		TIMER0	Moved from 0x40054000
0x400B8000	0x400B80FF		TIMER1	New: second timer instance
0x400C0000	0x400C0FFF		HSTX	High-Speed TX (DVI/HDMI)
0x400D0000	0x400D0FFF		QMI	QSPI Memory Interface (replaces XIP SSI)
0x400E0000	0x400E00FF	256 B	BOOTRAM	Boot scratch RAM
0x400F0000	0x400F0FFF		TRNG	True Random Number Generator (xorshift LFSR)
0x400F8000	0x400F8FFF		SHA-256	SHA-256 accelerator (register storage)
0x40100000	0x401000FF		POWMAN	Power Manager (VREG, BOD, AON timer)

Start Address	End Address	Size	Region	Note
0x40108000	0x401080FF	32 KB	TICKS	9 tick generators
0x40120000	0x401200FF		OTP controller	OTP programming registers
0x40130000	0x40137FFF		OTP data	8192 rows x 16-bit (32-bit aligned readout)
0x40140000	0x4014003F	48 B	CORESIGHT	CoreSight trace (register storage)
0x40158000	0x4015801F		GLITCH	Glitch detector (register storage)
0x40160000	0x401600FF		ACCESSCTRL	Peripheral access control (default all-access)
0x50400000	0x50400FFF	16 B	PIO2	New: third PIO block
0xD0000100	0xD000012F		CLINT	RISC-V only: mtime, mtimecmp, MSIP
0xD00001C0	0xD00001CF		Hart launch	RISC-V only: entry, SP, arg, launch trigger

4.3 4.3 Peripheral Base Addresses (Complete Table)

Base Address	Peripheral	IP Core	Emulation
0x40000000	SYSINFO	–	CHIP_ID (RP2040 or RP2350), PLATFORM=ASIC
0x40004000	SYSCFG	–	Full: NMI mask, proc config, debug force, mem power-down
0x40008000	Clocks	–	Full: 10 generators, FC0_RESULT from PLL config
0x4000C000	Resets	–	Full: RE-SET/RESET_DONE bitmask tracking
0x40014000	IO_BANK0	–	Full: 48 GPIO pins (30 on RP2040), edge/level IRQ
0x40018000	IO_QSPI	–	Stub: 6 QSPI pins, STATUS/CTRL + interrupt regs
0x4001C000	PADS_BANK0	–	Pad electrical control registers
0x40020000	PADS_QSPI	–	Stub: QSPI pad control
0x40024000	XOSC	–	Full: STATUS.STABLE + ENABLED
0x40028000	PLL_SYS	–	Full: CS.LOCK, PWR, FBDIV, PRIM
0x4002C000	PLL_USB	–	Full: CS.LOCK, PWR, FBDIV, PRIM
0x40030000	BUSCTRL	–	Full: BUS_PRIORITY, PERFSEL/PERFCTR (x4)
0x40034000	UART0	PL011	Full: DR, FR, IBRD/FBRD, LCR_H, CR, IFLS, IMSC, RIS/MIS, ICR, 16-deep RX FIFO, peripheral ID
0x40038000	UART1	PL011	Full: independent second instance

Base Address	Peripheral	IP Core	Emulation
0x4003C000	SPI0	PL022	Full: SSPCR0/1, SSPDR, SSPSR, SSPCPSR, SSPIMSC, RIS/MIS, ICR, 8-deep TX/RX FIFOs, device callbacks, peripheral ID
0x40040000	SPI1	PL022	Full: independent second instance
0x40044000	I2C0	DW_apb_i2c	Full: IC_CON, TAR, SAR, DATA_CMD, SCL timing, ENABLE, STATUS, 16-deep RX FIFO, up to 8 device callbacks, CLR_* regs, COMP_TYPE/VERSION
0x40048000	I2C1	DW_apb_i2c	Full: independent second instance
0x4004C000	ADC	–	Full: CS (AINSEL, START_ONCE, READY, RROBIN), RESULT, FCS (EN, SHIFT, LEVEL, EMPTY, FULL, OVER, UNDER), FIFO read, DIV, INTR/INTE/INTF/INTS. 5 channels (GPIO 26-29 + temp ~27C). 4-deep circular FIFO with 12->8 bit shift. Round-robin.
0x40050000	PWM	–	Full: 8 slices x (CSR, DIV, CTR, CC, TOP), EN, INTR/INTE/INTF/INTS

Base Address	Peripheral	IP Core	Emulation
0x40054000	Timer	–	Full: 64-bit us counter, 4 alarms, ARMED, PAUSE, INTR/INTE/INTF/INTS. Alarm comparison: (int32_t)(low - target) >= 0. Atomic 64-bit reads (TIMELR latches high).
0x40058000	Watchdog	–	Full: CTRL (bit 31 TRIGGER = reboot), LOAD, REASON (0=clean boot), TICK (RUNNING), SCRATCH0-7
0x4005C000	RTC	–	Full: CLKDIV_M1, SETUP_0/1, CTRL (LOAD strobe, ENABLE, ACTIVE), RTC_1/0 (running time), IRQ regs. Calendar rollover. Leap year.
0x40060000	ROSC	–	Full: CTRL (enable 0xFAB), STATUS (STABLE, ENABLED), RANDOMBIT (xorshift LFSR), FREQA/B, DIV, PHASE, COUNT
0x40064000	VREG_AND_CHIP_RESET		Full: VREG (EN, VSEL, ROK), BOD (EN, VSEL), CHIP_RESET (had_por, had_run, had_psm, W1C)
0x4006C000	TBMAN	–	Full: PLATFORM = 0x00000002 (ASIC)

Base Address	Peripheral	IP Core	Emulation
0x50000000	DMA	–	Full: 12 channels, 4 alias layouts, CTRL_TRIG (DATA_SIZE, INCR_READ/WRITE, CHAIN_TO, IRQ_QUIET, EN), INTR/INTE0/1/INTF0/1/INTS0/1, MULTI_CHAN_TRIGGER. Synchronous transfers.
0x50100000	USB DPRAM	–	Full: 4KB dual-port RAM, byte/halfword/word access, buf_ctrl (AVAILABLE bit 10, FULL bit 15, LEN bits 9:0)
0x50110000	USB Regs	–	Full: host enumeration simulation, CDC data bridge, SIE_STATUS/CTRL, BUFF_STATUS (W1C), INTR/INTE/INTF/INTS, multi-packet IN accumulation
0x50200000	PIO0	–	Full: 4 SMs, 32-word instr mem, 4-deep TX/RX FIFOs, all 9 opcodes, clock divider (16.8 fp), autopush/pull, force-exec, FSTAT/FLEVEL, INTR/INTE/INTF/INTS
0x50300000	PIO1	–	Full: independent second PIO block
0x50400000	PIO2	–	Full: third PIO block (RP2350 only)

Base Address	Peripheral	IP Core	Emulation
0xD0000000	SIO	–	Full: GPIO (IN/OUT/OE + atomic), CPUID, FIFO (32-deep per direction), hardware divider (per-core signed/unsigned, div-by-zero=0xFFFFFFFF, INT32_MIN/-1 wraps), 2 interpolators per core (ACCUM, BASE, CTRL_LANE, PEEK/POP/FULL), 32 spinlocks
0xE000E000	NVIC/SysTick/SCB	–	Full: per-core NVIC (enable, pending, priority, IABR), per-core SysTick (CSR, RVR, CVR, CALIB=0xC0002710), SCB (CPUID configurable, ICSR, VTOR, AIRCR with SYSRESETREQ, SCR, CCR, SHPR2, SHPR3). 4 priority levels (M0+) or 256 (M33).

All peripherals in the 0x40000000–0x50FFFFFF range support **atomic register aliases**:

Alias	Address Offset	Operation
Normal	+0x0000	Direct read/write
XOR	+0x1000	Atomic XOR: reg ^= value
SET	+0x2000	Atomic set: reg = value
CLR	+0x3000	Atomic clear: reg &= ~value

The SRAM alias at 0x21000000 is a **simple mirror** (not atomic). Atomic aliases are peripheral-only.

Chapter 5

Part 5: RP2040 ARM CPU Engine

5.1 CPU State Structure (cpu_state_t)

```
typedef struct {
    uint8_t  flash[FLASH_SIZE_MAX]; // 4MB XIP flash
    uint8_t  ram[RAM_SIZE];          // 264KB SRAM
    uint32_t r[16];                  // R0-R12, SP(R13), LR(R14), PC(R15)
    uint32_t xpsr;                    // Combined xPSR (APSR + IPSR + EPSR)
    uint32_t vtor;                    // Vector Table Offset Register
    uint32_t step_count;              // Instructions executed
    int      debug_enabled;           // -debug flag
    int      debug_asm;               // -asm flag
    uint32_t current_irq;             // Active exception vector number
    uint32_t primask;                 // PRIMASK (bit 0 = interrupt disable)
    uint32_t faultmask;               // FAULTMASK (bit 0 = mask all except NMI)
    uint32_t control;                 // CONTROL register (nPRIV bit 0, SPSEL bit 1)
} cpu_state_t;
```

5.2 Per-Core State (cpu_state_dual_t)

```
typedef struct {
    uint8_t  ram[CORE_RAM_SIZE];      // 132KB per-core RAM partition
    uint32_t r[16];                   // Per-core register file
    uint32_t xpsr, vtor;
    uint32_t step_count;
    int      debug_enabled, debug_asm;
    uint32_t current_irq;
    uint32_t primask, faultmask, control;

    // Exception nesting (per-core)
    uint32_t exception_stack[MAX_EXCEPTION_DEPTH]; // 8 deep
    int      exception_depth;

    // Core state
    int      core_id;                 // 0 or 1
    int      is_halted;                // Core stopped
    int      is_wfi;                   // Sleeping (WFI/WFE)
    uint32_t exception_sp;             // Saved SP for exception handling
}
```

```
int    in_handler_mode; // True if in ISR
} cpu_state_dual_t;
```

5.3 5.3 Instruction Dispatch

The CPU engine uses a two-level dispatch:

1. **256-entry dispatch table** indexed by instruction bits [15:8]. Each entry is a function pointer to the appropriate Thumb-1 handler. This provides O(1) dispatch for all 16-bit instructions.
2. **64K-entry decoded instruction cache** (direct-mapped by PC). Stores the pre-decoded handler function pointer + raw 16-bit instruction for each PC. On a cache hit, both `mem_read16()` and the dispatch table lookup are skipped. Invalidated on RAM writes; flash/ROM entries are never invalidated.
3. **32-bit instruction path**: If bits [15:11] of the first halfword match a 32-bit encoding prefix (0b11101, 0b11110, 0b11111), the second halfword is fetched and `thumb32_step()` is called.
4. **JIT basic block cache** (-jit): 16384-entry cache of consecutive instruction sequences (max 64 instructions per block). Blocks terminate at branches, 32-bit instructions, WFI/WFE, SVC, BKPT, CPS, or the 64-instruction limit. Block execution batches cycle counting and skips per-instruction PC validation, interrupt checks, and dispatch lookups. ~1.5x speedup over icache alone.

5.4 5.4 Instruction Set (Complete)

5.4.1 Thumb-1 (16-bit) Instructions

Category	Instructions	Details
Data processing	ADD, ADC, SUB, SBC, RSB, MOV, MVN, MUL	ADCS/SBCS set all NZCV flags. RSBS = negate (0 - Rn). MUL sets NZ, preserves CV.
Comparison	CMP (imm8, reg), CMN, TST	Set flags without writing result
Logical	AND, ORR, EOR, BIC	Set NZ flags
Shift/Rotate	LSL (imm5, reg), LSR (imm5, reg), ASR (imm5, reg), ROR (reg)	Shift by 32: LSL=0, LSR=0 with C=bit31, ASR=sign-fill. Shift by 0: no change.

Category	Instructions	Details
Load (word)	LDR (imm5, SP+imm8, PC+imm8, reg), LDRH, LDRB, LDRSH, LDRSB	PC-relative loads word-aligned. LDRSH/LDRSB sign-extend.
Store	STR (imm5, SP+imm8, reg), STRH, STRB	
Load/Store multiple	LDMIA (Rn!, {reglist}), STMIA (Rn!, {reglist})	LDMIA writeback NOT applied if base in reglist (loaded value wins per ARMv6-M).
Stack	PUSH {reglist, LR}, POP {reglist, PC}	POP PC with EXC_RETURN: SP updated BEFORE exception return processing.
Branch	B (imm11), Bcc (imm8, 14 conditions), BX (reg), BLX (reg), BL (32-bit)	Taken branch: 2 cycles. Not taken: 1. BX/BLX: 3. BL: 4.
System	SVC (imm8), BKPT (imm8), NOP, SEV, WFI, WFE, YIELD, CPSIE i, CPSID i	SVC triggers SVCall (exception 11). BKPT triggers HardFault (not halt). CPSID/CPSIE set/clear PRIMASK.
Misc	SXTH, SXTB, UXTH, UXTB, REV, REV16, REVSH, ADR (PC-relative imm)	
32-bit	BL (T1), MSR, MRS, DSB, DMB, ISB	Barriers are NOPs in emulator (no reordering).

5.4.2 Thumb-2 (32-bit) Instructions (Available in M33 Mode)

Category	Instructions	Details
Move immediate	MOVW (16-bit imm to Rd), MOVT (16-bit imm to top half)	MOVW zero-extends. MOVT preserves bottom half.
Add/Sub wide	ADDW (12-bit imm), SUBW (12-bit imm)	
Divide	SDIV (signed), UDIV (unsigned)	Div-by-zero returns 0.
Multiply	MUL (T2), MLA (multiply-accumulate), MLS (multiply-subtract)	

Category	Instructions	Details
Long multiply	SMULL, UMULL, SMLAL, UMLAL	64-bit result in RdLo:RdHi
Bit field	BFI (insert), BFC (clear), UBFX (unsigned extract), SBFX (signed extract)	
Bit manipulation	CLZ (count leading zeros)	
Branch	B.W (T4, 24-bit offset), Bcc.W (T3, conditional 20-bit offset)	
Table branch	TBB (byte offsets), TBH (halfword offsets)	
Load/Store	Wide LDR.W, STR.W (imm12, imm8, register shifted), LDRB.W, STRB.W, LDRH.W, STRH.W, LDRSB.W, LDRSH.W	
Data processing	Shifted register forms of AND, ORR, EOR, BIC, ADD, SUB, ADC, SBC, RSB, CMP, CMN, TST, TEQ	
Load/Store multiple	LDMIA.W, STMIA.W, LDMDB, STMDB (decrement before)	

5.5 Exception Handling

5.5.1 Exception Entry

1. Push 8-word stack frame: {R0, R1, R2, R3, R12, LR, return_PC, xPSR}.
2. Set LR to EXC_RETURN magic value:
 - 0xFFFFFFF1: Return to handler mode, MSP
 - 0xFFFFFFF9: Return to thread mode, MSP
 - 0xFFFFFFF0: Return to thread mode, PSP
3. Update xPSR IPSR field with new exception number.
4. Push vector number onto exception nesting stack (depth 8).
5. Load PC from vector table: `mem_read32(VT0R + vector_num * 4)`.

5.5.2 Exception Return

1. Detect EXC_RETURN in BX or POP PC (value 0xFFFFFFF*).
2. **Tail-chaining:** Before unstacking, check for pending higher-priority exceptions. If found, skip unstack/restack and switch handler directly.
3. **Late-arriving:** During stacking (before handler fetch), if a higher-priority IRQ becomes pending, switch to that handler instead.
4. Pop 8-word stack frame, restore registers.
5. FAULTMASK auto-cleared on exception return.

5.5.3 Double-Fault Lockup

If HardFault occurs while the HardFault handler is active (exception depth shows HardFault already on stack), the core enters **lockup**: `is_halted = 1`, `PC = 0xFFFFFFFF`. This matches real Cortex-M0+ behavior (DDI 0484C, Section 2.1.3).

5.5.4 Interrupt Delivery Conditions

An interrupt is delivered when ALL of the following are true:

1. The IRQ is enabled in `NVIC_ISER`.
2. The IRQ is pending in `NVIC_ISPR`.
3. `PRIMASK == 0` (interrupts not globally disabled).
4. `FAULTMASK == 0` (exceptions not masked by `FAULTMASK`).
5. The IRQ's priority is **less than** (higher priority than) the currently active exception's priority.
6. For M33 mode: `BASEPRI == 0` OR the IRQ's priority is less than `BASEPRI`.

5.5.5 Priority Model

Exception	Vector	Fixed Priority
Reset	1	-3 (highest)
NMI	2	-2
HardFault	3	-1
SVCall	11	Configurable (SHPR2 bits [31:30])
PendSV	14	Configurable (SHPR3 bits [23:22])
SysTick	15	Configurable (SHPR3 bits [31:30])
IRQ 0–25	16–41	Configurable (IPR0–7, 2 bits per IRQ)

Cortex-M0+: 4 priority levels (2 bits, in positions [7:6]). Cortex-M33: 256 priority levels (8 bits).

5.6 5.6 Timing Model

Cycle costs per Cortex-M0+ TRM (DDI 0484C):

Instruction Class	Cycles	Notes
ALU (ADD, SUB, MOV, CMP, AND, ORR, etc.)	1	Single-cycle
LDR, STR (all variants)	2	Memory access
LDM, STM	1 + N	N = register count
PUSH, POP	1 + N	N = register count

Instruction Class	Cycles	Notes
BX, BLX	3	Register indirect branch
BL (32-bit)	4	Link branch
B taken (conditional or unconditional)	2	Pipeline flush
B not taken (Bcc only)	1	No flush
MUL	1	Single-cycle multiplier on M0+

The cycle accumulator converts CPU cycles to microseconds:

```

timing_tick(cycles):
    cycle_accumulator += cycles
    while cycle_accumulator >= cycles_per_us:
        cycle_accumulator -= cycles_per_us
        timer_tick(1)    // 1 microsecond elapsed
        rtc_tick(1)

```

SysTick counts in **raw CPU cycles** (not microseconds), per ARM specification.

Chapter 6

Part 6: RP2350 RISC-V CPU Engine

6.1 6.1 CPU State (rv_cpu_state_t)

```
typedef struct {
    uint32_t x[32];           // x0 hardwired to 0, x1=ra, x2=sp, ...
    uint32_t pc;              // Program counter
    uint32_t csr[4096];       // Full 12-bit CSR address space
    uint64_t cycle_count;      // mcycle (64-bit)
    uint64_t instret_count;    // minstret (64-bit)

    // Atomic reservation (LR.W / SC.W)
    uint32_t lr_reservation;   // Reserved address
    int      lr_valid;         // 1 if reservation active

    // Hazard3 stack protection
    uint32_t stack_base;      // mstack_base CSR (0xBC0)
    uint32_t stack_limit;     // mstack_limit CSR (0xBC1)
    int      stack_guard_enabled;

    int      in_trap;         // Currently handling a trap
    int      hart_id;         // 0 or 1
    int      is_halted;
    int      is_wfi;
    uint32_t step_count;
    int      debug_enabled;
    void     *bus;            // rv_membus_state_t*
    void     *icache;         // rv_icache_t*
} rv_cpu_state_t;
```

6.2 6.2 Standard CSRs

Address	Name	Access	Description
0x300	mstatus	RW	Machine status (MIE bit 3, MPIE bit 7, MPP bits 12:11)
0x301	misa	RO	ISA description: 0x40100115 (RV32, I+M+A+C)

Address	Name	Access	Description
0x304	mie	RW	Machine interrupt enable (MSIE bit 3, MTIE bit 7, MEIE bit 11)
0x305	mtvec	RW	Machine trap vector (bit 0: 0=direct, 1=vectored)
0x340	mscratch	RW	Machine scratch register
0x341	mepc	RW	Machine exception PC (saved on trap entry)
0x342	mcause	RW	Machine cause (bit 31: interrupt. bits 30:0: code)
0x343	mtval	RW	Machine trap value (faulting address or instruction)
0x344	mip	RO	Machine interrupt pending (computed from CLINT hardware state)
0xB00	mcycle	RW	Machine cycle counter (low 32 bits)
0xB02	minstret	RW	Machine instructions retired (low 32 bits)
0xB80	mcycleh	RW	mcycle high 32 bits
0xB82	minstreth	RW	minstret high 32 bits
0xF11	mvendorid	RO	Vendor ID (0 = non-commercial)
0xF12	marchid	RO	Architecture ID
0xF13	mimpid	RO	Implementation ID
0xF14	mhartid	RO	Hart ID (0 or 1)

6.3 Hazard3-Specific CSRs

Address	Name	Access	Description
0xBE0	meie0	RW	External IRQ enable [31:0]. Updates CLINT ext_enable.
0xBE1	meie1	RW	External IRQ enable bits [51:32] (20 bits).
0xFE0	meip0	RO	External IRQ pending bits [31:0]. Computed as <code>clint.ext_pending & csr[MEIE0]</code> .
0xFE1	meip1	RO	External IRQ pending bits [51:32].
0xFE2	mlei	RO	Lowest enabled pending IRQ number. Returns 0xFFFFFFFF if none.
0xBE2	meiea	RW	External IRQ array enable access
0xFE4	meipa	RO	External IRQ array pending access
0xBE4	meifa	RW	External IRQ array force
0xBE6	meicontext	RW	External IRQ context save/restore
0xBC0	mstack_base	RW	Stack lower bound (hardware stack protection)
0xBC1	mstack_limit	RW	Stack upper bound

6.4 Trap Handling

6.4.1 Trap Entry (`rv_trap_enter`)

1. Save pc to mepc.
2. Save cause to mcause (bit 31 set for interrupts).
3. Save trap value to mtval (faulting address or instruction word).
4. Save `mstatus.MIE` to `mstatus.MPIE`.
5. Clear `mstatus.MIE` (disable interrupts during handler).
6. Set `mstatus.MPP` to M-mode.
7. Compute handler address from mtvec:
 - Direct mode (bit 0 = 0): `pc = mtvec_base`
 - Vectored mode (bit 0 = 1) + interrupt: `pc = mtvec_base + cause * 4`
 - Vectored mode + exception: `pc = mtvec_base`

6.4.2 Trap Return (MRET)

1. Restore pc from mepc.
2. Restore mstatus.MIE from mstatus.MPIE.
3. Set mstatus.MPIE = 1.

6.4.3 Trap Causes

Code	Name	Trigger
0	Instruction address misaligned	Branch/JAL to odd address
1	Instruction access fault	Fetch from unmapped address
2	Illegal instruction	Unrecognized opcode
3	Breakpoint	EBREAK instruction
4	Load address misaligned	LH/LW on unaligned address
5	Load access fault	Load from unmapped address
6	Store address misaligned	SH/SW on unaligned address
7	Store access fault	Store to unmapped address
11	Environment call from M-mode	ECALL instruction
0x80000003	Machine software interrupt	CLINT MSIP
0x80000007	Machine timer interrupt	CLINT mtime >= mtimecmp
0x8000000B	Machine external interrupt	CLINT ext_pending & ext_enable

6.5 CLINT Interrupt Controller

Memory-mapped at 0xD0000100 in SIO space:

Offset	Register	Description
0x00	MTIME_LO	64-bit free-running timer, low word
0x04	MTIME_HI	High word
0x08	MTIMECMP0_LO	Hart 0 timer compare, low word
0x0C	MTIMECMP0_HI	High word
0x10	MTIMECMP1_LO	Hart 1 timer compare, low word
0x14	MTIMECMP1_HI	High word
0x20	MSIP0	Hart 0 software interrupt (bit 0 only)

Offset	Register	Description
0x24	MSIP1	Hart 1 software interrupt (bit 0 only)

Timer behavior:

- `rv_clint_tick(cycles)` accumulates `cycles`. When `cycle_accum >= cycles_per_us`, `mtime` increments by the elapsed microseconds.
- Timer interrupt: `mtime >= mtimecmp[hart_id]` sets `MIP_MTIP` (bit 7 of `mip`).
- `mtimecmp` initialized to `UINT64_MAX` (no immediate interrupt on boot).

Interrupt delivery priority: **MEIP > MSIP > MTIP**.

6.6 RISC-V Instruction Cache

64K-entry direct-mapped cache for flash/ROM instruction fetches:

```
typedef struct {
    uint32_t tag;    // PC address (0 = invalid)
    uint32_t instr;  // Cached instruction word
    uint8_t size;    // 2 = compressed, 4 = 32-bit
} rv_icache_entry_t;
```

- Indexed by `(pc >> 1) & 0xFFFF`.
- Only caches addresses in ROM (`< 0x8000`) and flash (`0x10000000–0x10FFFFFF`).
- SRAM instruction fetches bypass the cache.
- Observed hit rates: 99.97%+ on real firmware boot code.

6.7 ROM Function Table

The RISC-V bootrom provides an SDK-compatible ROM function table:

Address	Function	Table Code	Native Implementation
0x0300	<code>rom_table_lookup</code> –		RISC-V code: iterates [<code>code</code> , <code>addr</code>] entries
0x0400	<code>memcpy</code>	'CP' (0x4350)	Byte-by-byte copy via <code>rv_membus</code>
0x0404	<code>memset</code>	'ST' (0x5453)	Byte fill via <code>rv_membus</code>
0x0408	<code>memcpy4</code>	–	Word-aligned copy
0x040C	<code>memset4</code>	–	Word-aligned fill
0x0410	<code>popcount32</code>	'PC' (0x5043)	<code>__builtin_popcount</code>
0x0414	<code>clz32</code>	'ZL' (0x5A4C)	<code>__builtin_clz</code> (32 if zero)
0x0418	<code>ctz32</code>	'ZT' (0x5A54)	<code>__builtin_ctz</code> (32 if zero)

Address	Function	Table Code	Native Implementation
0x041C	reverse32	'RV' (0x5652)	Bit reversal
0x0420	flash_enter_xip	–	No-op
0x0424	flash_exit_xip	–	No-op
0x0428	flash_range_erase	'RE' (0x4552)	memset(flash+offset, 0xFF, count)
0x042C	flash_range_program	'RP' (0x5052)	Byte copy from SRAM to flash
0x0430	reboot	'RB' (0x4252)	Sets is_halted = 1
0x0434	set_stack	–	Sets SP (x2)

All stubs are RET instructions (JALR x0, x1, 0). `rv_rom_intercept()` checks the PC before each `rv_cpu_step()` call; if it matches a stub address, the operation is performed natively in C and the PC is set to x1 (return address).

6.8 Picobin IMAGE_DEF Parser

The RP2350 bootrom uses **picobin blocks** (IMAGE_DEF) in the first 4 KB of flash to configure boot:

6.8.1 Block Structure

0xFFFFDED3	Block start marker
[item 0]	Typed item (packed 32-bit words)
[item 1]	
...	
[LAST item]	Type 0xFF, terminates item list
block_loop_offset	Offset to next block (0 = single block loop)
0xAB123579	Block end marker

6.8.2 Item Encoding

Each item is one or more 32-bit words. The first word encodes:

Bits	Field	Description
[7:0]	type	Item type identifier
[15:8]	size	Total size in 32-bit words (including this header word)
[31:16]	flags	Type-specific flags

6.8.3 Item Types

Type	Name	Size	Description
0x42	IMAGE_TYPE	1	Image type flags: bits[3:0]=type (1=EXE, 2=DATA), bits[10:8]=CPU (0=ARM, 1=RISC-V), bits[14:12]=chip (0=RP2040, 1=RP2350)
0x44	ENTRY_POINT	3	Word 1: initial PC. Word 2: initial SP.
0x03	VECTOR_TABLE	2	Word 1: vector table address (ARM only).
0xFF	LAST	variable	End sentinel. Low byte of size field = total block word count.

6.8.4 Example: RISC-V IMAGE_DEF

As found in MicroPython Pico 2 RISC-V firmware at flash offset 0x0014:

```

0xFFFFDED3      Start marker
0x11010142      IMAGE_TYPE: type=0x42, size=1, flags=0x1101
                  (EXE | CPU_RISCV | CHIP_RP2350)
0x00000344      ENTRY_POINT: type=0x44, size=3, pad=0
0x10000036      Entry PC
0x20082000      Entry SP (top of 520KB SRAM)
0x000004FF      LAST item
0x00000000      Block loop offset (single block)
0xAB123579      End marker

```

Chapter 7

Part 7: RP2350 Peripherals

7.1 7.1 TICKS (0x40108000)

9 tick generators, each with CTRL and CYCLES registers (stride 8 bytes):

Index	Generator	Default
0	PROC0	Enabled
1	PROC1	Enabled
2	TIMER0	Enabled
3	TIMER1	Enabled
4	WATCHDOG	Enabled
5	RISCV	Disabled
6	REFTICK	Disabled
7	ADC	Disabled
8	Reserved	Disabled

7.2 7.2 POWMAN (0x40100000)

Power manager with VREG/BOD control:

Offset	Register	Default	Description
0x00	VREG_CTRL	0xB1	Voltage regulator (EN, VSEL=1.1V)
0x04	VREG_STATUS	–	Returns VREG_CTRL
0x08	BOD_CTRL	0x91	Brown-out detector
0x0C	BOD_STATUS	–	Returns BOD_CTRL
0x10	STATE	0x0F	Power domain state (all domains on)
0x50	TIMER_LO	0	AON timer low word
0x54	TIMER_HI	0	AON timer high word
0x60	INTE	0	Interrupt enable
0x64	INTF	0	Interrupt force
0x68	INTS	0	Interrupt status

7.3 7.3 QMI (0x400D0000)

QSPI Memory Interface (replaces XIP SSI on RP2350):

Offset	Register	Description
0x00	DIRECT_CSR	Direct mode control (BUSY=0, EN=1)
0x04	DIRECT_TX	Direct TX data
0x08	DIRECT_RX	Direct RX data
0x0C	M0_TIMING	Memory 0 timing configuration
0x10	M0_RFMT	Memory 0 read format
0x14	M0_RCMD	Memory 0 read command (default: 0x03 = standard SPI read)
0x18	M0_WFMT	Memory 0 write format
0x1C	M0_WCMD	Memory 0 write command
0x20–0x30	M1_*	Memory 1 configuration (same layout)
0x34–0x53	ATRANS[0–7]	Address translation registers

7.4 7.4 OTP (0x40120000 / 0x40130000)

- **Controller** (0x40120000): 32 general-purpose registers for OTP programming control.
- **Data readout** (0x40130000): 8192 rows of 16-bit data, each at a 32-bit aligned address. Default: all 0xFFFF (unprogrammed).
- Read: `otp_data[row] = mem_read32(0x40130000 + row * 4) & 0xFFFF`
- Writes to data region are ignored (one-time programmable).

7.5 7.5 BOOTRAM (0x400E0000)

256-byte boot scratch RAM shared between bootrom and firmware. Supports byte, halfword, and word access. Initialized to zero.

7.6 7.6 TIMER1 (0x400B8000)

Second hardware timer instance with identical register layout to the RP2040 timer:

- 64-bit microsecond counter with 4 alarm comparators.
- Same signed alarm comparison: `(int32_t)(current_low - target) >= 0`.
- Ticked from the CLINT timer path (`rp2350_timer1_tick()`).

7.7 7.7 Other RP2350 Peripherals

Peripheral	Address	Description
TRNG	0x400F0000	True Random Number Generator (xorshift32 LFSR pseudo-random)
SHA-256	0x400F8000	SHA-256 accelerator (register storage, accepts writes)
HSTX	0x400C0000	High-Speed TX for DVI/HDMI (register storage, status ready)
GLITCH	0x40158000	Glitch detector (8-word register storage)
CORESIGHT	0x40140000	CoreSight trace (16-word register storage)
ACCESSCTRL	0x40160000	Peripheral access control (64-word storage, default all-access)

Chapter 8

Part 8: SIO and Inter-Core Communication

8.1 8.1 SIO Base (0xD0000000)

Offset Range	Function	Description
0x000	CPUID	Returns current core ID (0 or 1). RP2350 returns 0x00000002.
0x004	GPIO_IN	GPIO input values (30 pins on RP2040, 32 on RP2350)
0x008	GPIO_HI_IN	QSPI GPIO input (RP2040: returns 0x3E). RP2350: GPIO pins 32–47 input.
0x010–0x02C	GPIO_OUT/OE + SET/CLR/XOR	GPIO output and output enable with atomic variants
0x030–0x04C	GPIO_HI_OUT/OE (RP2350)	GPIO pins 32–47 output/enable (RP2350 only)
0x050–0x05C	FIFO	Inter-core FIFO: WR/RD for each direction
0x060–0x078	Hardware Divider	Per-core signed/unsigned division. Div-by-zero: result=0xFFFFFFFF. INT32_MIN/-1: wraps to INT32_MIN.
0x080–0x0FF	Interpolators	2 per core: ACCUM0/1, BASE0/1/2, CTRL_LANE0/1, PEEK/POP/FULL
0x100–0x17C	Spinlocks	32 hardware spinlocks. Read: acquire (returns 0x80000001 if free, 0 if locked). Write: release.
0x1C0–0x1CF	Hart Launch (RP2350 RV)	Entry, SP, arg, launch trigger for hart 1

8.2 8.2 Inter-Core FIFO

32-entry circular FIFO per direction:

- **Core 0** writes to FIFO0_WR (0xD00000050); **Core 1** reads from FIFO0_RD (0xD00000058).
- **Core 1** writes to FIFO1_WR (0xD00000054); **Core 0** reads from FIFO1_RD (0xD0000005C).
- `fifo_try_push()` signals NVIC IRQ 15/16 (SIO_IRQ_PROC0/1) for the receiving core.
- FIFO_ST register: VLD (bit 0 = data available to read), RDY (bit 1 = space to write).
- Used by Pico SDK for Core 1 launch protocol: Core 0 writes {0, 0, 1, VTOR, SP, entry} to FIFO.

8.3 RP2350 Hart Launch Mailbox

For RISC-V mode, hart 1 is launched via SIO registers:

Offset	Register	Description
0x1C0	HART1_BOOT_ENTRY	Entry point PC for hart 1
0x1C4	HART1_BOOT_SP	Stack pointer (x2) for hart 1
0x1C8	HART1_BOOT_ARG	Argument (x10/a0) for hart 1
0x1CC	HART1_BOOT_LAUNCH	Write 1 to trigger hart 1 launch

The main execution loop polls `rv_membus_check_hart1_launch()` each iteration. When triggered, hart 1 is reset with the specified PC, SP, and argument.

Chapter 9

Part 9: Debugging

9.1 GDB Remote Debugging

9.1.1 Setup

```
# Terminal 1: Start emulator with GDB server
./bramble firmware.uf2 -gdb 3333

# Terminal 2: Connect GDB (ARM)
arm-none-eabi-gdb firmware.elf -ex "target remote :3333"

# Terminal 2: Connect GDB (RISC-V)
riscv32-unknown-elf-gdb firmware.elf -ex "target remote :3333"
```

9.1.2 Supported RSP Commands

Command	Description
?	Halt reason (returns T05 with thread ID)
g / G	Read / write all registers
p / P	Read / write single register by number
m addr,len / M addr,len:data	Read / write memory
c / s	Continue / single step
Z0 addr,len / z0 addr,len	Set / remove software breakpoint
Z1 addr,len / z1 addr,len	Set / remove hardware breakpoint
Z2 addr,len / z2 addr,len	Set / remove write watchpoint
Z3 addr,len / z3 addr,len	Set / remove read watchpoint
Z4 addr,len / z4 addr,len	Set / remove access watchpoint
vCont;c / vCont;s	Continue / step with thread selection
Hg thread / Hc thread	Set thread for register / continue operations
qfThreadInfo / qsThreadInfo	Enumerate threads (m1,2 for dual-core)
qSupported	Feature query (returns PacketSize=4096;swbreak+;hwbreak+)
qRcmd hex_cmd	Monitor command (conditional breakpoints)
D / k	Detach / kill

Command	Description
Ctrl-C	Interrupt execution

9.1.3 Architecture-Aware Registers

Mode	Register Count	Layout
ARM (M0+/M33)	17	R0–R15 (32-bit LE hex each) + xPSR
RISC-V (RV32)	33	x0–x31 (32-bit LE hex each) + PC

The GDB stub automatically detects the active architecture via `gdb_is_riscv` and uses the appropriate register layout.

9.1.4 Conditional Breakpoints

Set via `qRcmd` (monitor) commands:

```
(gdb) monitor cond 0 r0==0x1234      # BP 0: break when R0 == 0x1234
(gdb) monitor cond 1 r3!=0           # BP 1: break when R3 != 0
(gdb) monitor cond 2 *0x20000000==5  # BP 2: break when [0x20000000] == 5
(gdb) monitor uncond 0               # Remove condition from BP 0
```

Condition types: `rN==val`, `rN!=val`, `rN<val`, `rN>val`, `*addr==val`, `*addr!=val`.

9.1.5 Watchpoint Stop Replies

Watchpoint Type	Stop Reply
Write (Z2)	T05watch:ADDR;thread:N;
Read (Z3)	T05rwatch:ADDR;thread:N;
Access (Z4)	T05awatch:ADDR;thread:N;

9.1.6 Limits

- 16 breakpoint slots.
- 16 watchpoint slots.
- Watchpoints checked on every memory access (gated by `gdb.active` flag).
- During GDB operation: instruction limit (1B) disabled, JIT disabled.

9.2 9.2 Debug Output Flags

Flag	Effect
-debug	Enables <code>cpu.debug_enabled</code> for Core 0 / Hart 0: logs exceptions, IRQ delivery, peripheral register accesses, exception entry/return
-debug1	Enables debug for Core 1 / Hart 1
-debug-mem	Logs every unmapped peripheral read/write to <code>stderr</code> . Essential for finding which peripherals a firmware needs that aren't yet implemented.
-status	Periodic status output: PC, SP, FIFO counts, core state, instruction count

All diagnostic output goes to `stderr`. Only firmware UART/USB output appears on `stdout`. This enables:

```
./bramble firmware.uf2 > output.txt 2>debug.log
```

Chapter 10

Part 10: Storage and Persistence

10.1 10.1 Flash Persistence

```
./bramble firmware.uf2 -flash storage.bin
```

Startup behavior:

1. Firmware is loaded into `cpu.flash[]` (initialized to `0xFF`).
2. If `-flash` file exists, each 4 KB sector is compared:
 - Sectors modified by firmware loading (not all `0xFF`) are kept.
 - Sectors that are still `0xFF` in the loaded firmware are restored from the file (filesystem data from previous run).
3. `flash_persist_open()` opens the file for write-through access.

Runtime behavior:

- Every `flash_range_erase` and `flash_range_program` call immediately syncs to the flash file via `flash_persist_sync()` (using `fseek + fwrite` to the affected offset).
- The file handle is kept open for efficient seeks.

Exit behavior:

- Full flash image saved to file on normal exit.

10.2 10.2 SD Card

SPI-mode SD card emulation (SDHC protocol):

- CSD v2.0, CID (product name “BRMSD”, manufacturer `0xBR`).
- Commands: `CMD0` (`GO_IDLE`), `CMD8` (`SEND_IF_COND`), `CMD9` (`SEND_CSD`), `CMD10` (`SEND_CID`), `CMD12` (`STOP_TRANSMISSION`), `CMD13` (`SEND_STATUS`), `CMD16` (`SET_BLOCKLEN`), `CMD17` (`READ_SINGLE_BLOCK`), `CMD18` (`READ_MULTIPLE_BLOCK`), `CMD24` (`WRITE_BLOCK`), `CMD25` (`WRITE_MULTIPLE_BLOCK`), `CMD55` (`APP_CMD`), `CMD58` (`READ_OCR`), `ACMD41` (`SD_SEND_OP_COND`).
- Block addressing (SDHC): address in block units, not byte units.
- File-backed: loaded on init, dirty flag tracks changes, flushed periodically (~1M steps) and on cleanup.
- Attaches to SPI bus via `spi_attach_device()` callback.

10.3 10.3 eMMC

SPI-mode eMMC emulation:

- CMD1 initialization (no ACMD41), CSD_STRUCTURE=3.
- EXT_CSD via CMD8 (512 bytes).
- Sector addressing always (large storage).
- Product name “BRMMC”, default 128 MB.
- File-backed with same flush behavior as SD card.

10.4 10.4 FUSE Mount

```
./bramble firmware.uf2 -flash storage.bin -mount /tmp/pico-fs
```

- Mounts the flash FAT12/FAT16 filesystem as a host directory via libfuse3.
 - Thread-safe: fuse_flash_mutex serializes FUSE operations against emulator flash writes.
 - FUSE writes trigger flash_persist_sync() for immediate disk persistence.
 - Auto-scans cpu.flash[] for FAT BPB signature (0x55AA at offset 510 + valid geometry).
 - Build requirement: cmake .. -DENABLE_FUSE=ON.
-

Chapter 11

Part 11: Networking, WiFi, and Multi-Device

11.1 11.1 UART-to-TCP Bridge

```
./bramble firmware.uf2 -net-uart0 9999      # Server mode
./bramble firmware.uf2 -net-uart0-connect host:9999 # Client mode
```

- Non-blocking I/O with TCP_NODELAY for low-latency byte-at-a-time transfer.
- Server mode: accepts one client at a time; auto-reconnects on disconnect.
- When active, UART TX routes through bridge instead of stdout.
- Both UART0 and UART1 independently configurable.

11.2 11.2 Wire Protocol (Multi-Instance IPC)

```
# Instance 1
./bramble fw_sensor.uf2 -wire-uart0 /tmp/uart.sock
# Instance 2 (auto-negotiates connection)
./bramble fw_ctrl.uf2 -wire-uart0 /tmp/uart.sock
```

- Unix domain socket IPC between Bramble instances.
- Auto-negotiation: first instance creates socket (listen), second connects.
- Wire message protocol: 4-byte header {type, channel, length, reserved} + payload.
- Message types: WIRE_MSG_UART_DATA, WIRE_MSG_GPIO_PIN, WIRE_MSG_SPI_XFER.
- UART TX on one instance delivered as UART RX on the other.
- GPIO pin changes propagated in real-time.
- Handles partial SOCK_STREAM reads/writes correctly.

11.3 11.3 CYW43439 WiFi Emulation

```
./bramble firmware.uf2 -wifi      # Basic emulation
./bramble firmware.uf2 -wifi -tap tap0 # TAP bridge
```

- CYW43439 WiFi chip emulation via gSPI protocol.
- gSPI command word: [31]=write, [30:28]=function, [27:17]=address, [16:0]=size.
- Three bus functions:
 - Function 0 (BUS): FEEDBEAD magic, test register echo.

- Function 1 (BACKPLANE): chip ID window, register access.
- Function 2 (WLAN): WiFi data frames.
- PIO0 SM0 intercept: TX FIFO writes trigger gSPI command processing; RX FIFO reads return responses. FSTAT reports virtual FIFO status.
- Configurable fake scan results via `cyw43_add_scan_result()`.

11.3.1 TAP Bridge

- `-tap tap0` creates a TAP interface on the host and bridges CYW43 WLAN frames to it.
 - Auto-configures: 192.168.4.1/24 via `ioctl`, brings UP, enables IP forwarding, sets up NAT masquerade.
 - Cleanup on close: removes NAT rules, restores forwarding state.
 - Requires `sudo` (auto-escalated if needed).
-

Chapter 12

Part 12: Performance

12.1 12.1 ARM Instruction Cache

64K-entry direct-mapped decoded instruction cache:

- Indexed by PC[15:0] (lower 16 bits).
- Each entry: {handler_fn_ptr, raw_16bit_instruction}.
- On hit: skips mem_read16() + dispatch table lookup.
- Invalidation: on any RAM write, the corresponding cache entry is invalidated. Flash/ROM entries are never invalidated (immutable code).
- Statistics reported on exit: hit count, miss count, hit rate percentage.

12.2 12.2 ARM JIT Basic Block Compilation (-jit)

16384-entry basic block cache for consecutive instruction sequences:

- **Block compilation:** Starting from a flash/ROM PC, consecutive Thumb-1 instructions are compiled into a block (max 64 instructions). Each entry stores the handler pointer and pre-computed cycle cost.
- **Block terminators:** Conditional/unconditional branches, BX/BLX, POP PC, SVC, BKPT, CPS, WFI/WFE, CBZ/CBNZ, 32-bit instructions. Identified via a 256-bit terminal bitmap indexed by instr[15:8].
- **Block execution** (jit_execute): Iterates through the block's instruction array, calling each handler directly. Step count batched (single add after block). PC only written when needed. Interrupt check performed once per block instead of per-instruction.
- **Constraints:** Only compiles flash/ROM code (immutable). Single-instruction blocks fall back to normal dispatch (zero overhead). Invalidated alongside icache on RAM writes.
- **Performance:** ~1.5x speedup over instruction cache alone. ~2x over baseline.

12.3 12.3 RISC-V Instruction Cache

64K-entry direct-mapped cache for flash/ROM instruction fetches:

- Indexed by (pc >> 1) & 0xFFFF.
- Stores full instruction word (16 or 32 bit) + size tag.
- Only caches ROM (< 0x8000) and flash (0x10000000+) addresses.
- Avoids repeated rv_mem_read16() calls on consecutive fetches from the same code region.
- Observed: 99.97%+ hit rates on littleOS RISC-V boot code.

12.4 12.4 Host Threading

- One host pthread per emulated core.
 - Global mutex (“big lock”) for all shared state.
 - WFI/WFE: releases mutex, sleeps on condition variable (zero host CPU usage while idle).
 - `corepool_wake_cores()` broadcasts condvar on interrupt delivery.
 - Core pool registry: file-based at `/tmp/bramble-corepool.reg`. Each instance registers its PID and core count. `-cores auto` queries the registry to find available cores.
 - Thread quantum (`-thread-quantum N`): number of guest instructions executed per lock acquisition. Default 64. Lower = more responsive to interrupts but higher lock overhead.
-

Chapter 13

Part 13: Tested Firmware

13.1 13.1 RP2040 Firmware

Firmware	Version	Command	Status
MicroPython	v1.27.0	./bramble micropython.uf2 -stdin -clock 125 -flash mpy.bin	Boots to interactive REPL via USB CDC
CircuitPython	10.1.3	./bramble circuitpython.uf2 -stdin -clock 125	Boots, runs code.py via USB CDC
littleOS	Latest	./bramble littleos.uf2 -stdin -clock 125 -flash los.bin	Full shell, SageLang, dual-core supervisor
hello_world	–	./bramble hello_world.uf2	Prints “Hello from Bramble RP2040 Emulator!”
gpio_test	–	./bramble gpio_test.uf2	Toggles GPIO 25
timer_test	–	./bramble timer_test.uf2	Measures elapsed time
alarm_test	–	./bramble alarm_test.uf2	Tests timer alarms and interrupts
interrupt_test	–	./bramble interrupt_test.uf2	Exception handling and nesting

13.2 13.2 RP2350 Firmware

Firmware	Arch	Status
littleOS pico2	M33	171M instructions, boot code runs
littleOS pico2 riscv	RV32	248M instructions, picobin boot
MicroPython Pico 2 RV	RV32	Picobin boot, 416 instructions

Example commands:

```
./bramble littleos_pico2.uf2 -arch m33 -clock 150  
./bramble littleos_pico2_riscv.uf2 -arch rv32 -clock 150  
./bramble micropython_pico2_rv.uf2 -arch rv32 -clock 150
```

RP2350 firmwares load, detect architecture via picobin/UF2 family ID, and execute through boot code. Full interactive output requires USB CDC enumeration completion (in progress).

Chapter 14

Part 14: Design Decisions and Trade-Offs

14.1 14.1 Correctness Over Performance

- Per-core NVIC faithfully models shared interrupt lines filtered by independent enable masks.
- Signed alarm comparison `(int32_t)(current_low - target) >= 0` handles 32-bit timer wrap correctly.
- Exception nesting stack (depth 8) instead of single `current_irq` enables proper nested interrupt handling.
- ARMv6-M double-fault lockup detection halts the core rather than causing infinite Hard-Fault recursion.
- LDM with base in register list: writeback NOT applied (loaded value wins, per ARMv6-M spec).
- POP PC with EXC_RETURN: SP updated BEFORE exception return processing (critical for nested exceptions).

14.2 14.2 Synchronous DMA

DMA transfers execute immediately (synchronously) when triggered, not cycle-by-cycle. This is simpler to implement, deterministic, and sufficient for all tested firmware. Firmware that depends on DMA pacing timers for transfer throttling is not supported.

14.3 14.3 Immediate PIO Execution

PIO state machines step once per main loop iteration. Clock dividers use 16.8 fixed-point accumulators. This is sufficient for UART, SPI, and I2C PIO programs. Very high-speed PIO programs (e.g., VGA timing) may not match real hardware timing precisely.

14.4 14.4 Tri-Architecture Shared Peripheral Bus

All three architectures share the same peripheral bus implementation in `membus.c`. The RP2350 modes add overlay routing:

- **RV32:** Memory goes through `rv_membus.c` first (520 KB SRAM, ROM, CLINT, RP2350 peripherals). Unhandled addresses fall through to `mem_read32()/mem_write32()`.
- **M33:** The ARM membus is augmented with `membus_rp2350_mode = 1`, which enables RP2350 SYSINFO, RP2350 peripheral routing, ROM write suppression, and 520 KB SRAM via `mem_set_ram_ptr()`.

- **M0+:** Standard RP2040 membus (264 KB SRAM, RP2040 peripherals only).

14.5 14.5 Output Separation

- All firmware output (UART TX, USB CDC data) goes to `stdout`.
- All emulator diagnostics (boot messages, debug output, statistics) go to `stderr`.
- This enables clean piping: `./bramble firmware.uf2 > output.txt` captures only firmware output.
- All runtime `printf` calls in peripheral/CPU code are gated behind `cpu.debug_enabled` flag.

14.6 14.6 Flash Initialized to 0xFF

The `cpu.flash[]` array is initialized to `0xFF` (erased state) before firmware loading, matching real hardware behavior. This is critical for flash persistence sector detection (firmware sectors have non-`0xFF` content; filesystem sectors may be all-`0xFF` if unused).

Chapter 15

Part 15: Repository Structure

```
bramble/
├─ CMakeLists.txt      Build config (v0.43.0)
├─ build.sh            Build script
├─ README.md, CHANGELOG.md, Bramble_Guide.md
├─
├─ src/
│   ├─ main.c          Entry point, CLI, loops
│   ├─ cpu.c            ARM CPU, dual-core, exceptions
│   ├─ instructions.c   65+ Thumb-1 handlers
│   ├─ thumb32.c        Full Thumb-2 handlers
│   ├─ membus.c         Memory bus routing
│   ├─ gpio.c           GPIO peripheral
│   ├─ timer.c          64-bit timer + 4 alarms
│   ├─ nvic.c           Per-core NVIC + SysTick
│   ├─ clocks.c         Clocks, XOSC, PLLs, Watchdog
│   ├─ adc.c            ADC (5ch, FIFO)
│   ├─ uart.c           Dual PL011 UART
│   ├─ spi.c            Dual PL022 SPI
│   ├─ i2c.c            Dual DW_apb_i2c
│   ├─ pwm.c            8-slice PWM
│   ├─ dma.c            12-ch DMA + chaining
│   ├─ pio.c            PIO (x3, 9 opcodes)
│   ├─ usb.c            USB + CDC bridge
│   ├─ rtc.c            RTC (calendar, leap year)
│   ├─ rom.c            ROM, soft-float/double
│   ├─ gdb.c            GDB RSP (ARM + RISC-V)
│   ├─ devtools.c       Dev tools + RP2350 stubs
│   ├─ netbridge.c      UART-to-TCP bridge
│   ├─ wire.c           Unix socket IPC
│   ├─ storage.c        Flash persistence
│   ├─ sdcard.c         SD card SPI emulation
│   ├─ emmc.c           eMMC SPI emulation
│   ├─ fatfs.c          FAT12/FAT16 driver
│   ├─ fuse_mount.c     FUSE mount
│   ├─ cyw43.c          CYW43 WiFi gSPI
│   ├─ tapif.c          TAP bridge
│   ├─ corepool.c       Host threading
│   ├─ uf2.c            UF2 loader
│   ├─ elf.c            ELF32 loader
│   ├─ w5500.c, bme280.c Device plugins
│   └─ rp2350_rv/
```


- |— rv_cpu.c Hazard3 RV32IMAC
 - |— rv_clint.c CLINT controller
 - |— rv_membus.c RP2350 memory bus
 - |— rv_bootrom.c RV bootrom + interception
 - |— rp2350_periph.c RP2350 peripherals
 - |— picobin.c Picobin parser
 - |— rp2350_arm/
 - |— m33_cpu.c M33 overlay
 - include/
 - |— emulator.h Core structures
 - |— *.h Per-peripheral headers
 - |— devtools.h Dev tools declarations
 - |— rp2350_rv/
 - |— rv_cpu.h RV CPU + CSR defs
 - |— rv_clint.h CLINT state
 - |— rv_membus.h Memory bus state
 - |— rv_bootrom.h ROM function addrs
 - |— rv_icache.h Instruction cache
 - |— rp2350_memmap.h Memory map constants
 - |— rp2350_periph.h Peripheral structs
 - |— picobin.h Picobin format defs
 - |— rp2350_arm/
 - |— m33_cpu.h M33 CPUID, BASEPRI
 - tests/
 - |— test_suite.c 300 automated tests
 - docs/
 - |— ROADMAP.md Roadmap + tracker

Chapter 16

Part 16: Version History

Version	Date	Key Features
v0.43.0	2026-03-21	littleOS RP2350 boot fixes: M33 520KB SRAM, RP2350 peripheral routing in ARM membus, ROM write suppression
v0.42.0	2026-03-20	Picobin IMAGE_DEF parser, MicroPython Pico 2 RV firmware tested, 4MB UF2 bounds
v0.41.0	2026-03-20	Cortex-M33 (-arch m33), BASEPRI, tri-architecture, 4 M33 tests
v0.40.0	2026-03-20	SDK-compatible ROM function table, RISC-V GDB, 4MB flash, 20 RV tests
v0.39.0	2026-03-20	Complete RP2350 peripherals, Hazard3 CSRs, PIO2, 48 GPIO, icache, semihosting, stack protection
v0.38.0	2026-03-20	CLINT, RP2350 memory bus, RISC-V bootrom, dual-hart, UF2/ELF auto-detect
v0.37.0	2026-03-20	RV32IMAC ISA complete (93 instructions), -arch rv32, tail-chaining, FAULTMASK, VREG
v0.36.0	2026-03-20	TAP auto-config, FAT auto-scan, FAT12, build.sh rewrite, zero warnings
v0.35.0	2026-03-20	Advanced devtools: symbols, callgraph, VCD, IRQ latency, script I/O, expect, heatmap
v0.32.0	–	Performance audit: SysTick O(1), NVIC CTZ, JIT 16K cache, ~33% throughput gain
v0.31.0	–	CYW43/Pico W, TAP bridge, JIT basic block compilation
v0.29.0	–	Per-core NVIC + SysTick, littleOS boots
v0.28.0	–	Host-threaded execution, -cores N, core pool
v0.27.0	–	Flash persistence, SD card, eMMC, FUSE mount

Version	Date	Key Features
v0.21.0	–	USB host enumeration, CDC bridge, MicroPython boots

Full history available in `CHANGELOG.md`.

Chapter 17

Part 17: External References

- [RP2040 Datasheet](#) — Primary hardware reference for all RP2040 peripherals
- [RP2350 Datasheet](#) — RP2350 hardware reference (Hazard3, M33, new peripherals)
- [Cortex-M0+ Technical Reference Manual \(DDI 0484\)](#) — Instruction timing, exception model
- [ARMv6-M Architecture Reference Manual \(DDI 0419\)](#) — ISA specification
- [RISC-V ISA Manual](#) — RV32IMAC instruction encoding
- [Pico SDK Documentation](#) — Firmware API reference
- [Pico SDK picobin.h](#) — Picobin block format
- [PL011 UART TRM \(DDI 0183\)](#) — UART register specification
- [PL022 SPI TRM \(DDI 0194\)](#) — SPI register specification
- [MicroPython](#) — MicroPython firmware downloads
- [CircuitPython](#) — CircuitPython firmware downloads
- [GDB Remote Serial Protocol](#) — RSP command reference
- [UF2 Specification](#) — UF2 format and family IDs